

NFDI4Earth

Josepha Schiller

2025-07-30

Project: NFDI4Earth Hackathon 11.2025

Goal: Explore data and run example machine learning analysis

Author: Josepha Schiller

Date: 30.07.2025

Last update: 19.09.2025

Provenance

This code adapts Ryo (2022) (https://github.com/masahiroryo/2022_IML_Agriculture/tree/main (https://github.com/masahiroryo/2022_IML_Agriculture/tree/main)).

Disclaimer: A GPT-5 agent was used to improve explanations and comments. All AI-assisted edits were reviewed and verified by the author.

1. Load libraries

Throughout this script we will rely on a number of packages from the R ecosystem. These packages provide helper functions for data manipulation, plotting, and machine learning. Below we **load** them at once so they are available later. You do **not** need to understand every package now; inline comments clarify each library's role.

```
library(tidyverse) # core collection of packages for data manipulation and visualisation
library(stringr)  # string processing functions used throughout the tidyverse
library(readr)    # efficient functions for reading flat files such as csv
library(patchwork) # combine multiple ggplot2 objects into one layout
library(reshape2) # additional tools for reshaping data (e.g. melt, dcast)
library(rlang)    # tidy evaluation tools; enables !!sym() in ggplot expressions
library(corrplot) # functions to visualise correlation matrices
library(car)      # `car` contains the `vif()` function for multicollinearity diagnostics

# Machine Learning specific Libraries
library(caret)    # wrapper for training and comparing many machine learning models
library(pdp)      # create partial dependence plots for model interpretation
library(vip)      # variable importance plots (including permutation importance)
library(iml)      # general interpretable machine-learning tools (not used directly here)
```

After loading the libraries, it can be helpful to print the current R session information. The `sessionInfo()` function returns details about your R version, operating system and loaded packages; this improves **reproducibility** when you share your analysis with others.

```
sessionInfo() # display versions of R and the loaded packages
```

```
## R version 4.3.2 (2023-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64 (64-bit)
## Running under: Windows 11 x64 (build 22621)
##
## Matrix products: default
##
##
## locale:
## [1] LC_COLLATE=English_United Kingdom.utf8
## [2] LC_CTYPE=English_United Kingdom.utf8
## [3] LC_MONETARY=English_United Kingdom.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United Kingdom.utf8
##
## time zone: Europe/Berlin
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] iml_0.11.3      vip_0.4.1      pdp_0.8.1      caret_6.0-94
## [5] lattice_0.21-9  car_3.1-2      carData_3.0-5  corrplot_0.92
## [9] rlang_1.1.3     reshape2_1.4.4 patchwork_1.2.0 lubridate_1.9.3
## [13] forcats_1.0.0   stringr_1.5.1  dplyr_1.1.4    purrr_1.0.2
## [17] readr_2.1.5     tidyr_1.3.1    tibble_3.2.1   ggplot2_3.5.1
## [21] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] tidymodels_1.2.1    timeDate_4032.109 fastmap_1.1.1
## [4] pROC_1.18.5         digest_0.6.35      rpart_4.1.21
## [7] timechange_0.3.0    lifecycle_1.0.4    survival_3.5-7
## [10] magrittr_2.0.3      compiler_4.3.2     sass_0.4.9
## [13] tools_4.3.2         utf8_1.2.4         yaml_2.3.8
## [16] data.table_1.15.4   knitr_1.46         plyr_1.8.9
## [19] abind_1.4-5         withr_3.0.0        Metrics_0.1.4
## [22] nnet_7.3-19         grid_4.3.2         stats4_4.3.2
## [25] fansi_1.0.6         colorspace_2.1-0   future_1.33.2
## [28] globals_0.16.3     scales_1.3.0       iterators_1.0.14
## [31] MASS_7.3-60         cli_3.6.2          rmarkdown_2.26
## [34] generics_0.1.3      rstudioapi_0.16.0  future.apply_1.11.2
## [37] tzdb_0.4.0          cachem_1.0.8       splines_4.3.2
## [40] parallel_4.3.2      vctrs_0.6.5        hardhat_1.3.1
## [43] Matrix_1.6-1.1      jsonlite_1.8.8     hms_1.1.3
## [46] listenr_0.9.1       foreach_1.5.2      gower_1.0.1
## [49] jquerylib_0.1.4     recipes_1.0.10     glue_1.7.0
## [52] parallelly_1.37.1   codetools_0.2-19   stringi_1.8.3
## [55] gtable_0.3.5        munsell_0.5.1      pillar_1.9.0
## [58] htmltools_0.5.8.1   ipred_0.9-14       lava_1.8.0
## [61] R6_2.5.1            evaluate_0.23      backports_1.4.1
## [64] bslib_0.7.0         class_7.3-22       Rcpp_1.0.12
## [67] checkmate_2.3.1     nlme_3.1-163       prodlim_2023.08.28
## [70] xfun_0.43           pkgconfig_2.0.3    ModelMetrics_1.2.2.2
```

2. Load data

In this section we read the dataset. The **Air Quality Index (AQI)** dataset is stored as a comma-separated values (CSV) file. You will need to adapt the path in `setwd()` to point to the directory where your copy of the dataset resides.

```
setwd("D:/Josi/PhD/Events/Kurse/2025_NFDI4Earth/2025_ML-Intro_Kurs/Hackathon_planning/aqi_data_kaggle") # set the working directory so R knows where to look for the data; change this to your own path
list.files() # inspect the files in the directory to confirm the dataset is present
```

```
## [1] "aqi_dataset.csv" "Readme.txt"
```

```
dta = read.table(file = "aqi_dataset.csv", header = TRUE, sep = ",") # read the CSV file into a data frame called dta
```

When reading in data, always check that the file loaded correctly and that the separator (`sep`) matches the file format. Using `header = TRUE` tells R to use the first row of the CSV file as column names.

3. Data exploration

We start with the exploratory data analysis (EDA) following the DIG framework. This includes getting an idea about what data is included in our dataset.

1. Describe the dataset
2. Introspection: brainstorm research questions
3. Goal setting: define a research question

3.1 Describe the dataset

What are the column variable names?

```
colnames(dta) # returns a vector with all column names in the data frame
```

```
## [1] "temperature"      "humidity"          "wind_speed"
## [4] "traffic_density"  "industrial_activity" "AQI"
```

Show one sample value from each column.

```
dta %>%
  summarise(across(everything(), ~ .x[!is.na(.x)][1])) # for each column, pick the first non-missing observation
```

```
##   temperature humidity wind_speed traffic_density industrial_activity   AQI
## 1      28.48    70.89      7.49           10             83.67 149.46
```

We can also look at the top rows.

```
dta %>% head() # display the first six rows to get a sense of the data
```

```
##   temperature humidity wind_speed traffic_density industrial_activity   AQI
## 1      28.48    70.89      7.49           10             83.67 149.46
## 2      24.03    65.24     14.13            9             49.93 164.90
## 3      29.53    55.93     15.29            7             82.52 155.39
## 4      35.66    74.91      9.91            3             31.72 105.21
## 5      23.36    60.99      8.38            3             45.26 118.21
## 6      23.36    53.03     14.88            5             37.80  66.10
```

We can look at the data structure.

```
dta %>% str() # show the structure: column names, types and preview of values
```

```
## 'data.frame':   14235 obs. of  6 variables:
## $ temperature   : num  28.5 24 29.5 35.7 23.4 ...
## $ humidity      : num  70.9 65.2 55.9 74.9 61 ...
## $ wind_speed    : num  7.49 14.13 15.29 9.91 8.38 ...
## $ traffic_density : num  10 9 7 3 3 5 7 7 NA 1 ...
## $ industrial_activity: num  83.7 49.9 82.5 31.7 45.3 ...
## $ AQI           : num  149 165 155 105 118 ...
```

How is the data quality?

Missing values per column:

```
colSums(is.na(dta)) # count the number of missing (NA) values in each column
```

```
##      temperature      humidity      wind_speed      traffic_density
##           283           284           280           142
## industrial_activity      AQI
##           142              0
```

or

```
dta %>%
  summarise(across(everything(), ~ sum(is.na(.x) | .x == ""), .names = "missing_{.col}")) # another way to count missing or empty values per column
```

```
##   missing_temperature missing_humidity missing_wind_speed
## 1           283           284           280
##   missing_traffic_density missing_industrial_activity missing_AQI
## 1           142              142              0
```

Data types for each column

```
sapply(dta, class) # returns the class (e.g., numeric, factor) of each column
```

```
##      temperature      humidity      wind_speed      traffic_density
##      "numeric"        "numeric"        "numeric"        "numeric"
## industrial_activity      AQI
##      "numeric"        "numeric"
```

We can also look at the summary statistics for this dataset. What to look for? Missing values, realistic ranges in variables.

```
summary(dta) # provides min, median, mean and max for numeric columns
```

```
## temperature      humidity      wind_speed      traffic_density
## Min.      :-2.46      Min.      : 2.15      Min.      :-3.400      Min.      : 1.000
## 1st Qu.:20.33      1st Qu.: 49.80      1st Qu.: 7.930      1st Qu.: 3.000
## Median :25.00      Median : 60.12      Median : 9.990      Median : 6.000
## Mean      :25.01      Mean      : 60.05      Mean      : 9.995      Mean      : 5.566
## 3rd Qu.:29.71      3rd Qu.: 70.17      3rd Qu.:12.060      3rd Qu.: 8.000
## Max.      :52.48      Max.      :127.19      Max.      :21.180      Max.      :10.000
## NA's      :283      NA's      :284      NA's      :280      NA's      :142
## industrial_activity      AQI
## Min.      :-33.15      Min.      : -75.17
## 1st Qu.: 36.55      1st Qu.: 82.92
## Median : 50.44      Median :121.33
## Mean      : 50.23      Mean      :122.09
## 3rd Qu.: 64.02      3rd Qu.:159.29
## Max.      :124.91      Max.      :531.03
## NA's      :142
```

Open question 1 Is it possible to have negative wind speed, negative industrial activities, and negative air quality? **Open question 2** Is it possible to have humanity higher than 100?

How much percent of the data are missing?

```
# Count total rows
n <- nrow(dta) # total number of rows in the dataset

col_missing <- colSums(is.na(dta)) # number of missing values per column

col_missing_pct <- (col_missing / n) * 100 # compute missing values as percentage of total rows

col_missing_pct # print the percentages for each column
```

```
##      temperature      humidity      wind_speed      traffic_density
##      1.9880576      1.9950825      1.9669828      0.9975413
## industrial_activity      AQI
##      0.9975413      0.0000000
```

It is important to keep in mind that for the machine learning analysis, we cannot use data rows that have NA values. That's why we can check how many rows remain after removing NA values.

```
dta %>% na.omit() %>% head() # remove any rows containing at least one missing value
```

```
## temperature humidity wind_speed traffic_density industrial_activity AQI
## 1      28.48      70.89      7.49      10      83.67 149.46
## 2      24.03      65.24      14.13      9      49.93 164.90
## 3      29.53      55.93      15.29      7      82.52 155.39
## 4      35.66      74.91      9.91      3      31.72 105.21
## 5      23.36      60.99      8.38      3      45.26 118.21
## 6      23.36      53.03      14.88      5      37.80 66.10
```

Check for duplicates.

```
# Check for duplicated rows
sum(duplicated(dta)) # returns the count of exact duplicate rows
```

```
## [1] 0
```

```
# View duplicated rows (if any)
dta[duplicated(dta), ] # print rows that are duplicates for inspection
```

```
## [1] temperature      humidity      wind_speed
## [4] traffic_density      industrial_activity AQI
## <0 rows> (or 0-length row.names)
```

Visual eploration

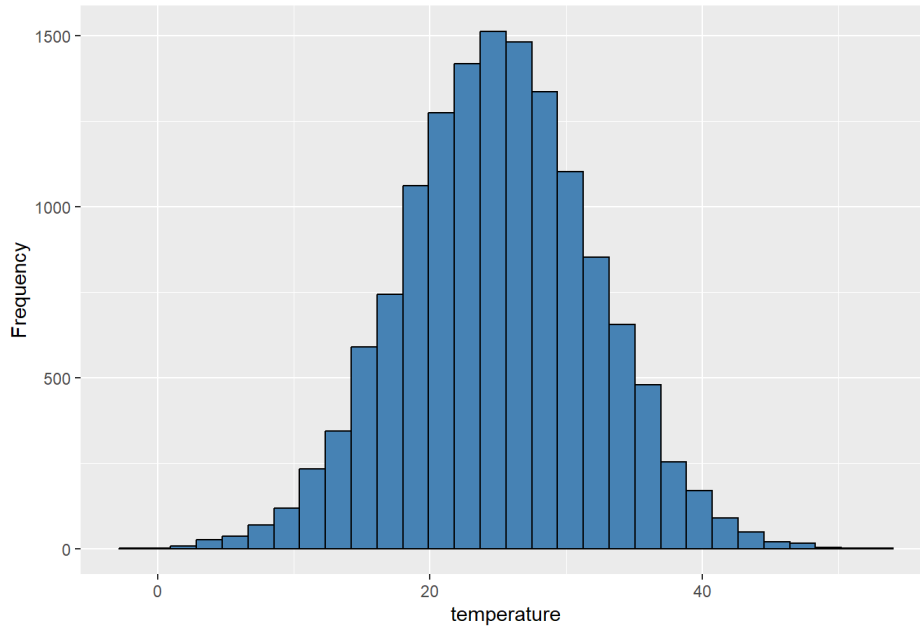
Histograms allow to see the range, distribution, and outliers.

```
df <- dta

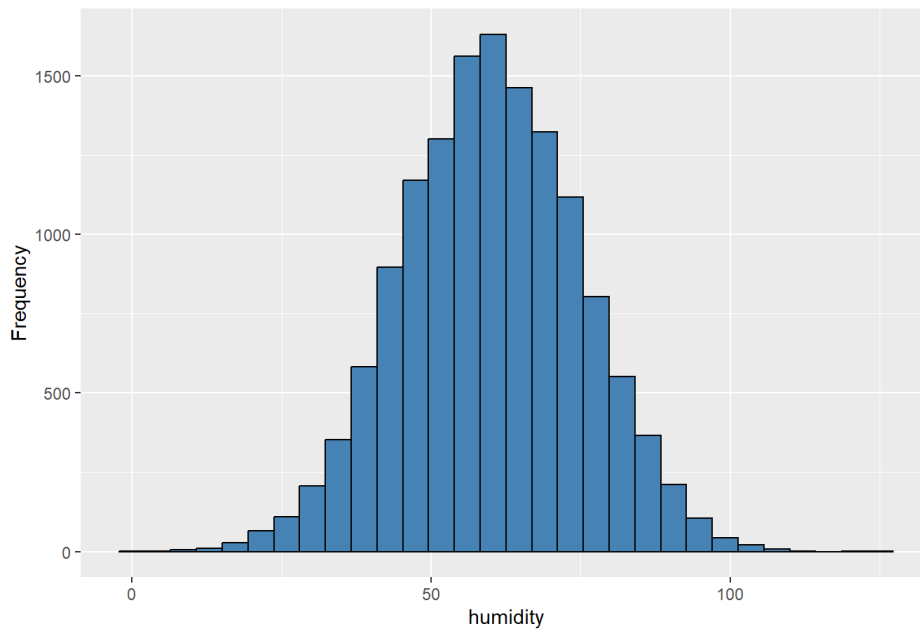
for (col in names(df)) {
  if (is.numeric(df[[col]])) {
    p <- ggplot(df, aes(x = !!sym(col))) +
      geom_histogram(bins = 30, fill = "steelblue", color = "black") + # 30 bins provides a reasonable balance of detail
      labs(title = paste("Histogram of", col), x = col, y = "Frequency")
    print(p)
  }
}
```

only produce histograms for numeric variables
dynamically map the column name using !!sym() from rLang
print each histogram within the loop

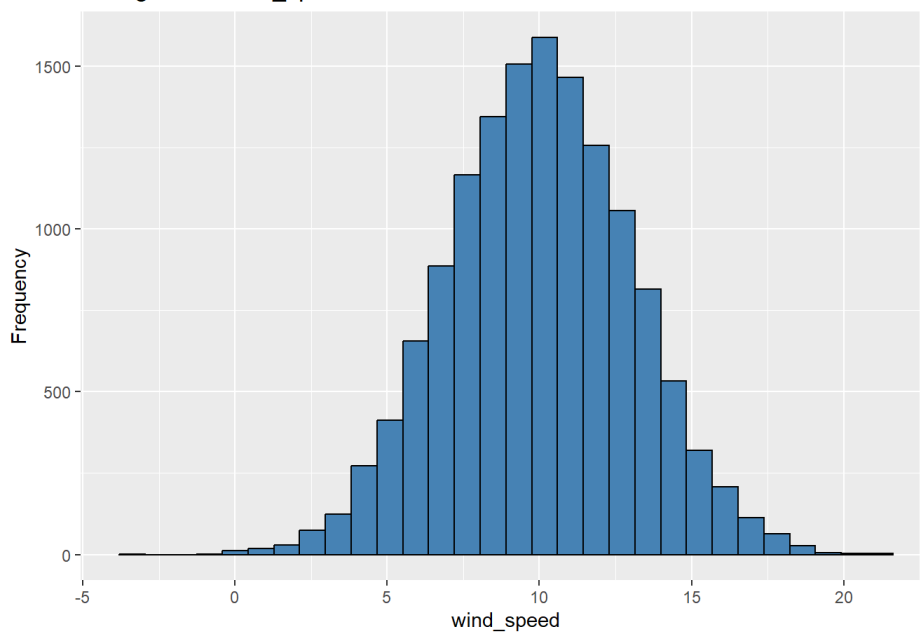
Histogram of temperature

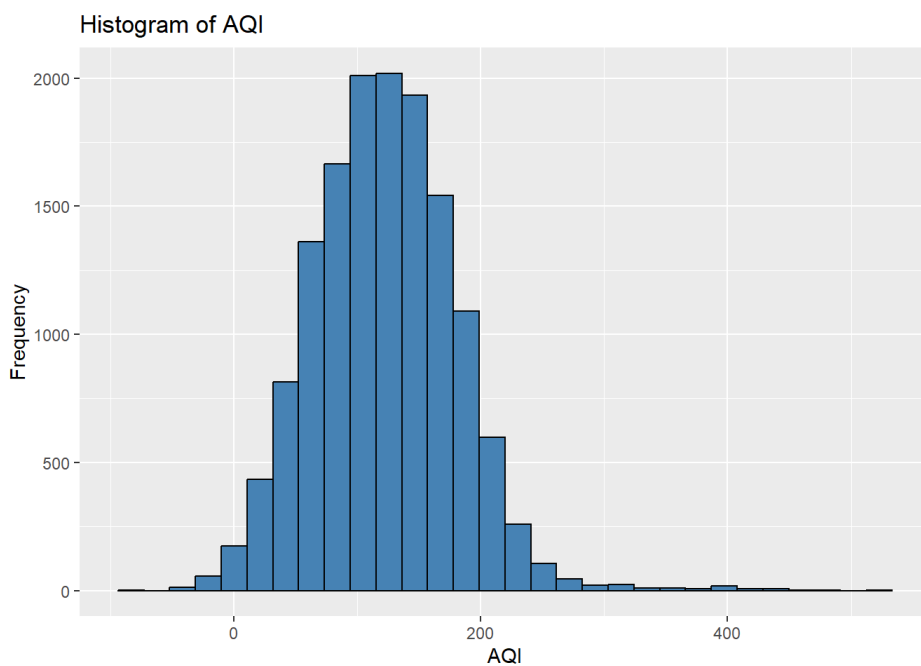
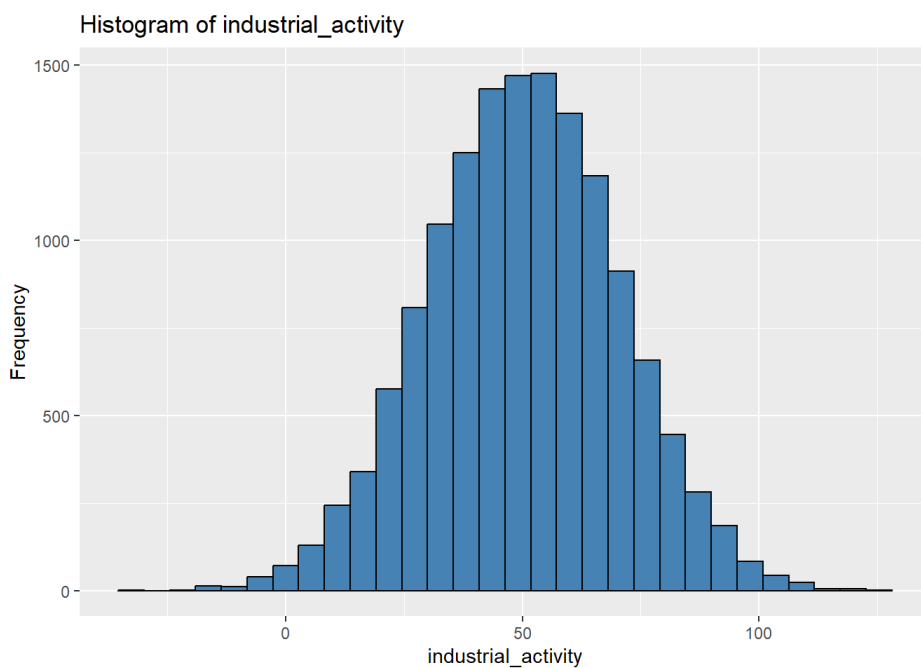
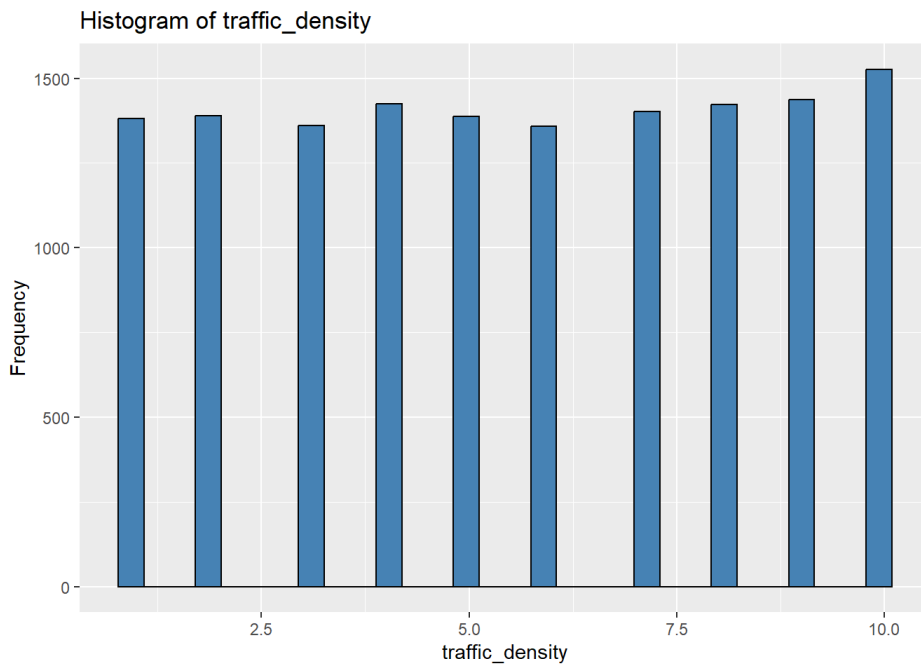


Histogram of humidity



Histogram of wind_speed





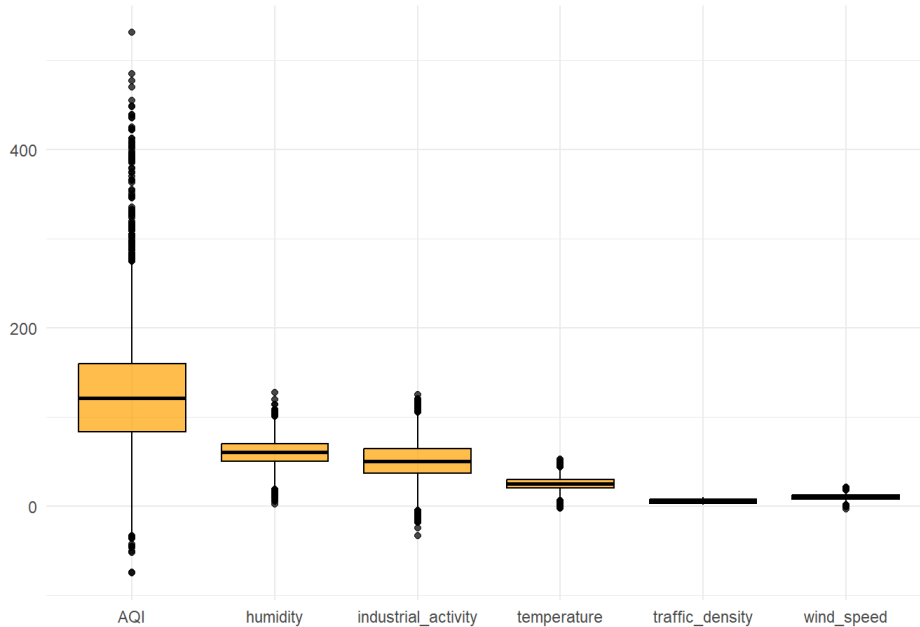
Box plots are nice because they give information about: Min, Q1, Median, Q3, Max. Outliers can be recognized beyond the whiskers.

```
# Long format of numeric columns
```

```
df_long <- df %>%
  dplyr::select(where(is.numeric)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") # reshape into two columns: variable and value

ggplot(df_long, aes(x = variable, y = value)) +
  geom_boxplot(fill = "orange", color = "black", alpha = 0.7) + # boxplots summarise distribution for each variable
  labs(title = "Boxplots per Variable", x = NULL, y = NULL) +
  theme_minimal()
```

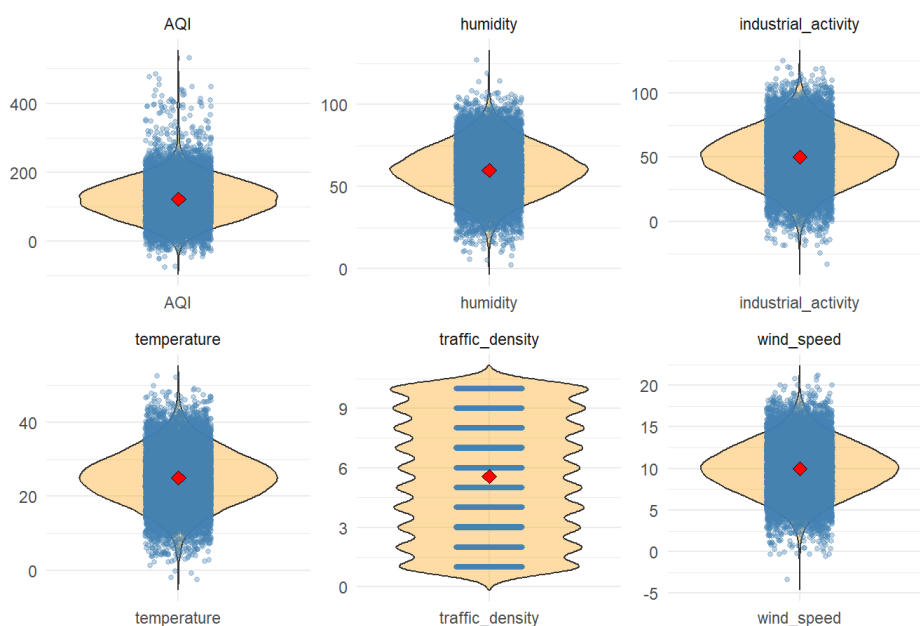
Boxplots per Variable



A very nice way to combine information from boxplots and histograms are violinplots.

```
ggplot(df_long, aes(x = variable, y = value)) +
  geom_violin(fill = "orange", alpha = 0.35, trim = FALSE) + # violin plots combine density estimation
  # n and range
  geom_jitter(width = 0.15, alpha = 0.35, size = 1, color = "steelblue") + # jittered points show individual observations
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.5, fill = "red") + # add a red diamond to denote the mean
  labs(title = "Violin + Jitter + Mean per Variable",
       x = NULL, y = NULL) +
  theme_minimal() +
  facet_wrap(~ variable, scales = "free") # facet each variable to its own panel
```

Violin + Jitter + Mean per Variable



3.2 Introspection

What research questions could possibly be answered?

After exploring the dataset and understanding its structure, the next step is to formulate potential research questions. The research question guides the direction of the analysis and help determine which variables are most relevant.

When defining the research question, one must ensure that the variables used for prediction are complete (no missing values). Why? Machine learning models cannot handle missing values directly. Variables with a large proportion of missing data are not suitable for use as predictors unless imputation or other strategies are applied.

Example research question: Predicting air quality Index based on the given variables

Now that we have a research question in mind, we can do more directed data exploration. Bivariate scatter plots with target variable "AQI" allow to detect links between target and predictor variables.

```
# Put data in Long format: predictors + AQI

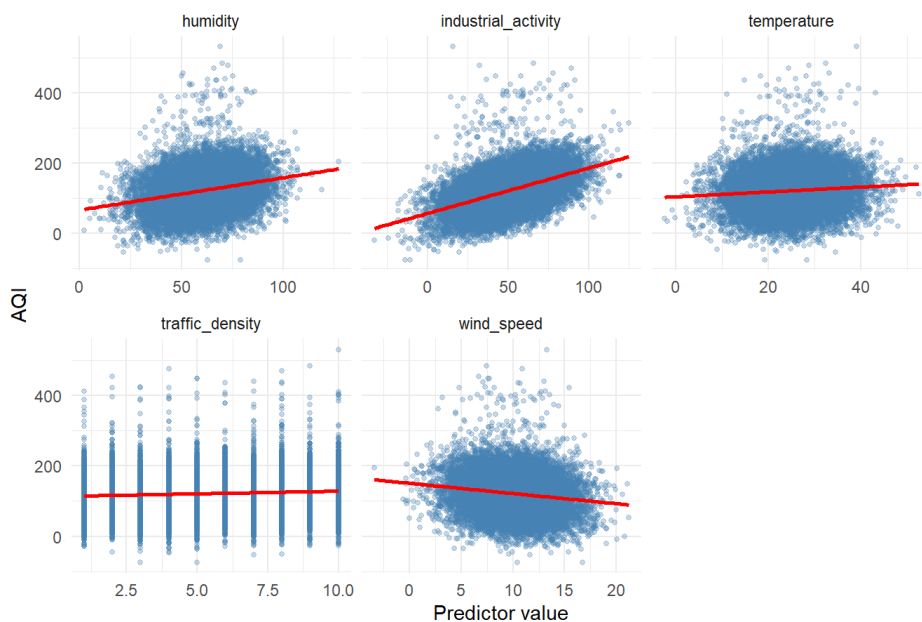
df_long <- df %>%
  select(where(is.numeric)) %>%
  pivot_longer(cols = -AQI, names_to = "variable", values_to = "value") # convert wide data to long form excluding AQI

# Scatter plots with regression lines, faceted by variable

ggplot(df_long, aes(x = value, y = AQI)) +
  geom_point(alpha = 0.3, size = 1, color = "steelblue") + # scatter plot of predictor vs AQI
  geom_smooth(method = "lm", se = FALSE, color = "red") + # add linear regression line without confidence band
  facet_wrap(~ variable, scales = "free_x", ncol = 3) + # separate plot for each predictor
  labs(title = "Scatter plots: AQI vs predictors",
       x = "Predictor value", y = "AQI") +
  theme_minimal()

## `geom_smooth()` using formula = 'y ~ x'
```

Scatter plots: AQI vs predictors



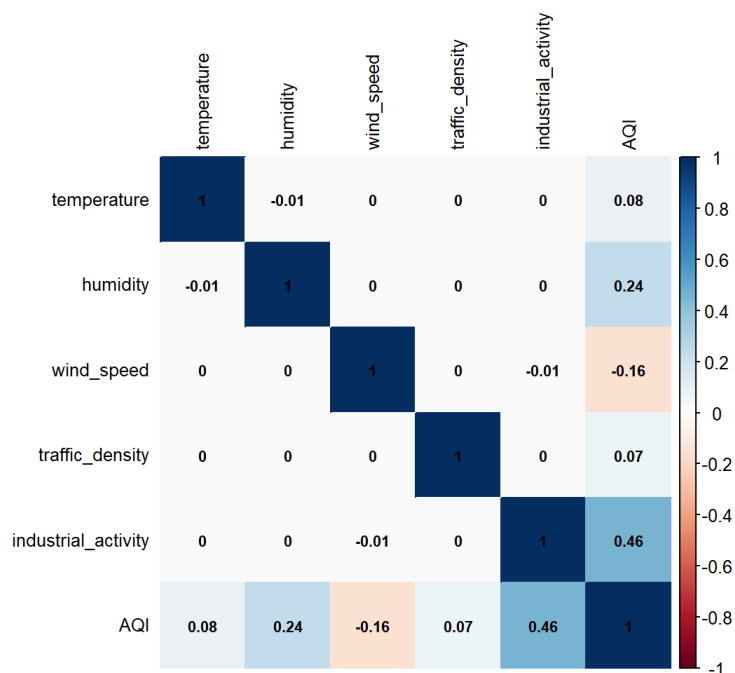
Use correlation matrix to test linear relationship between variables.

```
RQ1 <- df %>% na.omit() # remove rows containing missing values before computing correlations
cor_matrix <- cor(RQ1) # calculate the Pearson correlation matrix for all numeric variables
print(cor_matrix) # print the resulting correlation coefficients
```

```
##          temperature      humidity      wind_speed      traffic_density
## temperature      1.000000000 -0.005788864 -0.0015978915  0.0035279439
## humidity          -0.005788864  1.000000000  0.0044232063  0.0041360852
## wind_speed        -0.001597892  0.004423206  1.0000000000  0.0005963093
## traffic_density    0.003527944  0.004136085  0.0005963093  1.0000000000
## industrial_activity 0.002506157  0.003910368 -0.0055497503  0.0045158838
## AQI               0.083101400  0.239153746 -0.1550923725  0.0741438201
##          industrial_activity      AQI
## temperature      0.002506157  0.08310140
## humidity          0.003910368  0.23915375
## wind_speed        -0.005549750 -0.15509237
## traffic_density    0.004515884  0.07414382
## industrial_activity 1.000000000  0.45631254
## AQI               0.456312537  1.000000000
```

Draw a heatmap. Variables with correlation coefficients higher abs(0.7) can be considered es multi-collinear.

```
corrplot(cor_matrix, method = "color",
         addCoef.col = "black", # overlay the numeric correlation values on the heatmap
         tl.col = "black",     # text colour for variable labels
         tl.cex = 0.8,         # scale label size
         number.cex = 0.7)     # scale the printed correlation coefficients
```



We can also check for variable inflation factor (VIF) to better detect multi-collinearity.

```
# Fit a linear model with AQI as outcome and all predictors
model <- lm(AQI ~ temperature + humidity + wind_speed + traffic_density + industrial_activity, data = df)

# Compute VIF
vif_values <- vif(model) # variance inflation factor for each predictor
print(vif_values)        # VIF values > 5-10 may indicate problematic multi-collinearity
```

```
##          temperature      humidity      wind_speed      traffic_density
##          1.000055        1.000086        1.000053        1.000050
## industrial_activity
##          1.000073
```

Based on correlation and VIF, we don't expect multi-collinearity.

4. Data analysis

Task: Regression

4.1 Test and training data

Common splits are 80/20 or 70/30 splits. Training data are used to fit a model. Testing data are then used to test the model's performance.

```
set.seed(1) # set seed to make the sampling reproducible
train_test_split <- sample(1:nrow(RQ1), 0.8 * nrow(RQ1), replace = FALSE) # randomly select 80% of row indices for training
data_train <- RQ1[train_test_split, ] # training set
data_test <- RQ1[-train_test_split, ] # hold-out test set comprising the remaining rows
```

4.2 Choose cross-validation

Cross-validation (CV) is a resampling method used to validate a fitted model during the training phase. It is also used to select hyperparameters to get the best performance.

```
tc = trainControl(method = "cv" , number = 5) # define a 5-fold cross-validation strategy for model training
```

In 5-fold CV the training dataset is split into k=5 subsets (typically k = 5-10). Each fold is used once as validation data, while the remaining k-1 are used as training data to fit a model. By choosing CV the full model training process becomes more robust.

When combined with a training controller (e.g., trainControl in R's caret package), multiple hyperparameter values are tested automatically, and the best-performing set is selected based on cross-validation results. This removes the need for manual hyperparameter tuning.

4.3 Fit the models

Caret is a wrapper package that supports many different machine learning models. This makes comparison of different models easy. It is important to set a seed before fitting the model to ensure reproducibility.

Fit a linear regression model: This model uses organic matter as the response (target) variable and the remaining variables as the predictor variables. A formula can be set manually, too, to select only particular variables.

```
set.seed(123) # fix the random number seed to make model training reproducible
model.lm = caret::train(AQI ~ ., data=data_train, method="glmStepAIC", trControl=tc) # train a linear regression model with stepwise selection
```

```

## Start: AIC=88891.47
## .outcome ~ temperature + humidity + wind_speed + traffic_density +
##   industrial_activity
##
##               Df Deviance   AIC
## <none>          19141508 88891
## - traffic_density 1 19262156 88942
## - temperature    1 19333735 88974
## - wind_speed      1 19878783 89207
## - humidity        1 20710242 89552
## - industrial_activity 1 24938340 91114
## Start: AIC=88921.95
## .outcome ~ temperature + humidity + wind_speed + traffic_density +
##   industrial_activity
##
##               Df Deviance   AIC
## <none>          19232896 88922
## - traffic_density 1 19374908 88982
## - temperature    1 19406132 88995
## - wind_speed      1 19913611 89212
## - humidity        1 20688319 89533
## - industrial_activity 1 25239651 91205
## Start: AIC=88870.17
## .outcome ~ temperature + humidity + wind_speed + traffic_density +
##   industrial_activity
##
##               Df Deviance   AIC
## <none>          19136594 88870
## - traffic_density 1 19322791 88950
## - temperature    1 19335396 88955
## - wind_speed      1 19819017 89163
## - humidity        1 20693899 89526
## - industrial_activity 1 25154509 91167
## Start: AIC=88952.95
## .outcome ~ temperature + humidity + wind_speed + traffic_density +
##   industrial_activity
##
##               Df Deviance   AIC
## <none>          19303913 88953
## - traffic_density 1 19436159 89008
## - temperature    1 19509153 89040
## - wind_speed      1 19938568 89223
## - humidity        1 20786614 89573
## - industrial_activity 1 24958684 91111
## Start: AIC=88626.96
## .outcome ~ temperature + humidity + wind_speed + traffic_density +
##   industrial_activity
##
##               Df Deviance   AIC
## <none>          18590975 88627
## - traffic_density 1 18746032 88695
## - temperature    1 18782437 88711
## - wind_speed      1 19213951 88902
## - humidity        1 20095635 89279
## - industrial_activity 1 24537392 90958
## Start: AIC=111064.8
## .outcome ~ temperature + humidity + wind_speed + traffic_density +
##   industrial_activity
##
##               Df Deviance   AIC
## <none>          23854933 111065
## - traffic_density 1 24037818 111143
## - temperature    1 24094768 111168
## - wind_speed      1 24693289 111426
## - humidity        1 25747082 111865
## - industrial_activity 1 31211497 113888

```

Print the fitted linear model summary

```
model.lm
```

```
## Generalized Linear Model with Stepwise Feature Selection
##
## 10511 samples
##    5 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 8410, 8409, 8408, 8409, 8408
## Resampling results:
##
##    RMSE      Rsquared   MAE
##  47.65434   0.3059586   37.88469
```

Fit a decision tree model: decision trees use an if-else structure to make predictions. The model can tend to overfitting.

```
set.seed(123)
model.cart = caret::train(AQI ~ ., data=data_train, method="ctree", trControl=tc) # conditional inference tree from the party package
```

Print the model summary

```
model.cart
```

```
## Conditional Inference Tree
##
## 10511 samples
##    5 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 8410, 8409, 8408, 8409, 8408
## Resampling results across tuning parameters:
##
##   mincriterion  RMSE      Rsquared   MAE
##   0.01          53.53882  0.1915277  42.33944
##   0.50          50.16659  0.2422727  39.78460
##   0.99          49.71085  0.2467694  39.46864
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mincriterion = 0.99.
```

Fit a random forest model: makes use of several decision trees and averages the predictions among multiple trees. Typically, using a model that combines several trees ((ensemble models) makes more robust and better predictions than a single tree. Computation takes some minutes.

```
set.seed(123)
model.rf = caret::train(AQI ~ ., data=data_train, method="rf", trControl=tc) # random forest: ensemble of decision trees
```

Print the model summary

```
model.rf
```

```
## Random Forest
##
## 10511 samples
##    5 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 8410, 8409, 8408, 8409, 8408
## Resampling results across tuning parameters:
##
##   mtry  RMSE      Rsquared   MAE
##   2     48.74034  0.2748542  38.67819
##   3     48.88012  0.2720870  38.75398
##   5     49.09063  0.2673275  38.88348
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 2.
```

Fit a gradient boosting model: this model also uses several trees. However, it starts with a single tree and improves this tree's performance several times by reducing its error rate sequentially. Computation takes some minutes.

```

set.seed(123)
model.gbm = caret::train(AQI ~ ., data=data_train, method="gbm", trControl=tc,
                        tuneGrid = expand.grid(n.trees = (1:5)*500,          # number of boosting iterations
                                              interaction.depth = (1:5)*3, # maximum depth of each tree
                                              shrinkage = 0.1,             # Learning rate controlling contribution of eac
                                              n.minobsinnode = 10))         # minimum number of observations per terminal n
h tree
ode

```

Print the model summary

```
model.gbm
```

```

## Stochastic Gradient Boosting
##
## 10511 samples
##      5 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 8410, 8409, 8408, 8409, 8408
## Resampling results across tuning parameters:
##
##  interaction.depth  n.trees  RMSE      Rsquared  MAE
##      3              500     48.38738  0.2855118  38.42303
##      3             1000     48.87820  0.2734584  38.78918
##      3             1500     49.26010  0.2645957  39.04279
##      3             2000     49.63137  0.2565891  39.31362
##      3             2500     49.98441  0.2488950  39.55075
##      6              500     49.00536  0.2700961  38.84866
##      6             1000     49.95386  0.2491795  39.47236
##      6             1500     50.74257  0.2342010  40.08797
##      6             2000     51.41840  0.2223343  40.59008
##      6             2500     51.94877  0.2139442  40.99908
##      9              500     49.43229  0.2611354  39.14875
##      9             1000     50.56339  0.2390425  40.00109
##      9             1500     51.55363  0.2215525  40.70136
##      9             2000     52.36028  0.2090447  41.31827
##      9             2500     52.94340  0.2006856  41.73364
##     12              500     50.04796  0.2478532  39.63811
##     12             1000     51.37235  0.2241963  40.57329
##     12             1500     52.44614  0.2074486  41.33241
##     12             2000     53.17666  0.1973173  41.84766
##     12             2500     53.75889  0.1901119  42.26952
##     15              500     50.27465  0.2447474  39.78272
##     15             1000     51.89172  0.2171151  40.97092
##     15             1500     52.92229  0.2021747  41.75189
##     15             2000     53.62669  0.1934426  42.25215
##     15             2500     54.25468  0.1855003  42.72084
##
## Tuning parameter 'shrinkage' was held constant at a value of 0.1
##
## Tuning parameter 'n.minobsinnode' was held constant at a value of 10
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were n.trees = 500, interaction.depth =
## 3, shrinkage = 0.1 and n.minobsinnode = 10.

```

Now, we are done training the models. In the next step, we test the model performance using the test dataset

4.4 Test the model performance

In this step, we make use of the fitted models to make predictions with the test dataset.

```

pred.lm    <- predict(model.lm, data_test) # generate predictions from the linear model on the test set
pred.cart  <- predict(model.cart, data_test) # predictions from decision tree
pred.rf    <- predict(model.rf, data_test)  # predictions from random forest
pred.gbm   <- predict(model.gbm, data_test) # predictions from gradient boosting machine

```

We compare the relationship between the observations and the predictions of the test data. One popular measure is the R-squared value. It explains the proportion of variance in the response variable that is explained by using the predictor variables.

One easy way to compute this measure is to use the squared correlation coefficient.

```
# r2: obs vs pred
r2.lm <- cor(pred.lm, data_test$AQI)^2 %>% round(.,4) # square of Pearson correlation for Linear model
r2.cart <- cor(pred.cart, data_test$AQI)^2 %>% round(.,4) # R^2 for decision tree
r2.rf <- cor(pred.rf, data_test$AQI)^2 %>% round(.,4) # R^2 for random forest
r2.gbm <- cor(pred.gbm, data_test$AQI)^2 %>% round(.,4) # R^2 for gradient boosting
```

We can compare the performance of the different models by showing a table with r-squared values or we can also show it visually.

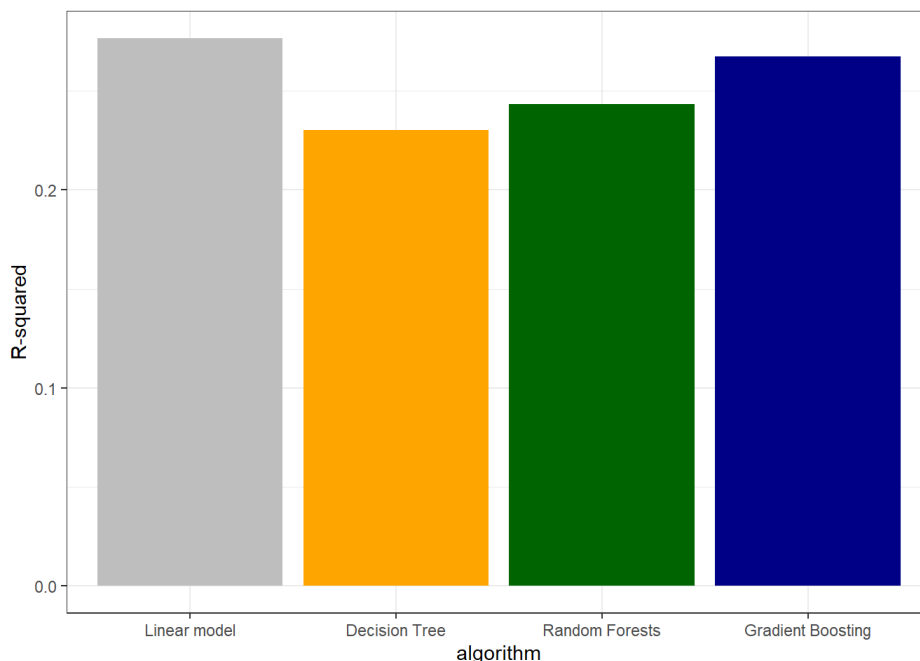
```
r2 <- data.frame(r2 = c(r2.lm, r2.cart, r2.rf, r2.gbm),
  algorithm = c("Linear model", "Decision Tree", "Random Forests", "Gradient Boosting") %>%
    factor(., levels = c("Linear model", "Decision Tree", "Random Forests", "Gradient Boosting")))

r2
```

```
##      r2      algorithm
## 1 0.2765    Linear model
## 2 0.2302    Decision Tree
## 3 0.2433    Random Forests
## 4 0.2673 Gradient Boosting
```

```
Fig02 <-
ggplot(r2, aes(x = algorithm, y = r2, fill = algorithm)) +
  geom_bar(stat = "identity") + # bar height represents R^2 for each model
  ylab("R-squared") +
  scale_fill_manual(values = c("Linear model" = "grey",
    "Decision Tree" = "orange",
    "Random Forests" = "darkgreen",
    "Gradient Boosting" = "darkblue")) +
  theme_bw() +
  theme(legend.position = "none") # hide redundant legend
```

Fig02 # display bar chart comparing model performances



Now we are done with testing the model performance, however we have no idea about how the fitted model actually looks like. E.g., what variables are important and in what relationship is a predictive variable with the response variable. Furthermore, we might have some errors in our models because of a wrong data preparation or variable use.

To get information about the models, we can make use of interpretable machine learning tools.

5 Interpretable machine learning

5.1 Variable importance

There are several interpretable machine learning tools available. A popular method is the variable importance method. There are several ways to get a variable importance. In this guide, we use the permutation variable importance. Computation takes some minutes.

Permutation importance works by **randomly permuting** the values of each feature and observing how much the model's predictive performance declines. If shuffling a feature leads to a large decrease in performance, the feature is considered important. In the `vip()` calls below:

- `metric = "rsq"` tells the function to evaluate the performance drop via cross-validated **R-squared** (i.e., how well the model explains variance in the data). Other metrics such as mean absolute error (`"mae"`) could be used instead.
- `nsim = 30` sets the number of permutations per feature. Using more simulations yields more stable importance estimates but takes longer to compute.
- `aesthetics = list(fill = ..., color = ...)` passes custom colours to the underlying **ggplot2** boxplot, allowing you to control the fill and outline colours of the importance bars.

These parameters do not affect the model itself; they only control how the importance is estimated and displayed.

```
# permutation-based feature importance
set.seed(123)
pvip_lm <- vip(model.lm, method = "permute", train = data_train, target = "AQI", metric = "rsq",
  pred_wrapper = predict, nsim = 30, geom = "boxplot",
  aesthetics = list(fill = "grey", color = "black")) +
  labs(title = "Linear model") + theme_bw()
# compute permutation importance for the linear model
set.seed(123)
pvip_cart <- vip(model.cart, method = "permute", train = data_train, target = "AQI", metric = "rsq",
  pred_wrapper = predict, nsim = 30, geom = "boxplot",
  aesthetics = list(fill = "orange", color = "black")) +
  labs(title = "Decision Tree") + theme_bw()
# importance for the decision tree

set.seed(123)
pvip_rf <- vip(model.rf, method = "permute", train = data_train, target = "AQI", metric = "rsq",
  pred_wrapper = predict, nsim = 30, geom = "boxplot",
  aesthetics = list(fill = "darkgreen", color = "black")) +
  labs(title = "Random Forests") + theme_bw()
# importance for the random forest

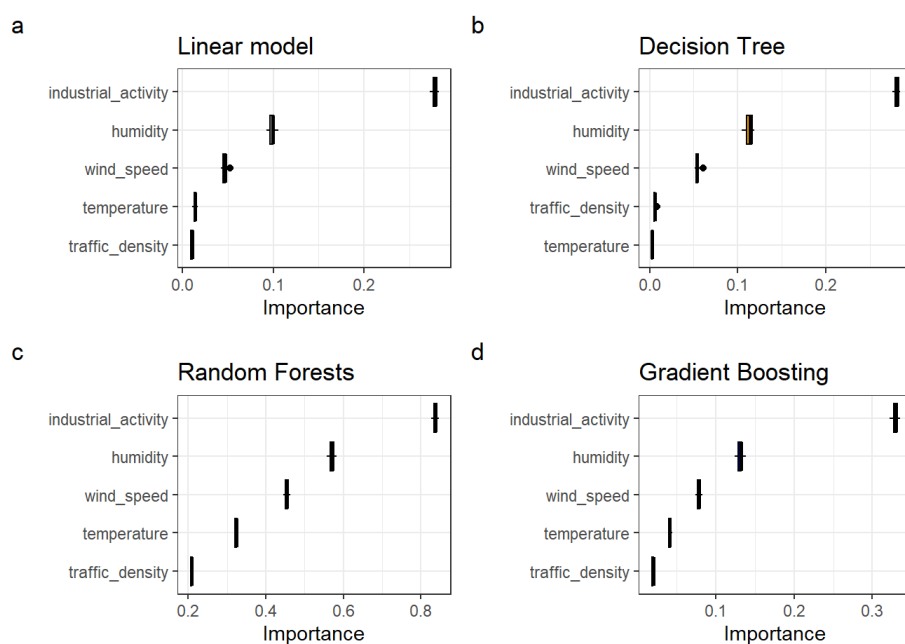
set.seed(123)
pvip_gbm <- vip(model.gbm, method = "permute", train = data_train, target = "AQI", metric = "rsq",
  pred_wrapper = predict, nsim = 30, geom = "boxplot",
  aesthetics = list(fill = "darkblue", color = "black")) +
  labs(title = "Gradient Boosting") + theme_bw()
# importance for the gradient boosting model
```

Combine the different plots.

```
plot_pvip_all <-
  pvip_lm + pvip_cart + pvip_rf + pvip_gbm
```

```
Fig03 <-
  plot_pvip_all +
  plot_annotation(tag_levels = 'a')
```

Fig03



5.1 Variable associations

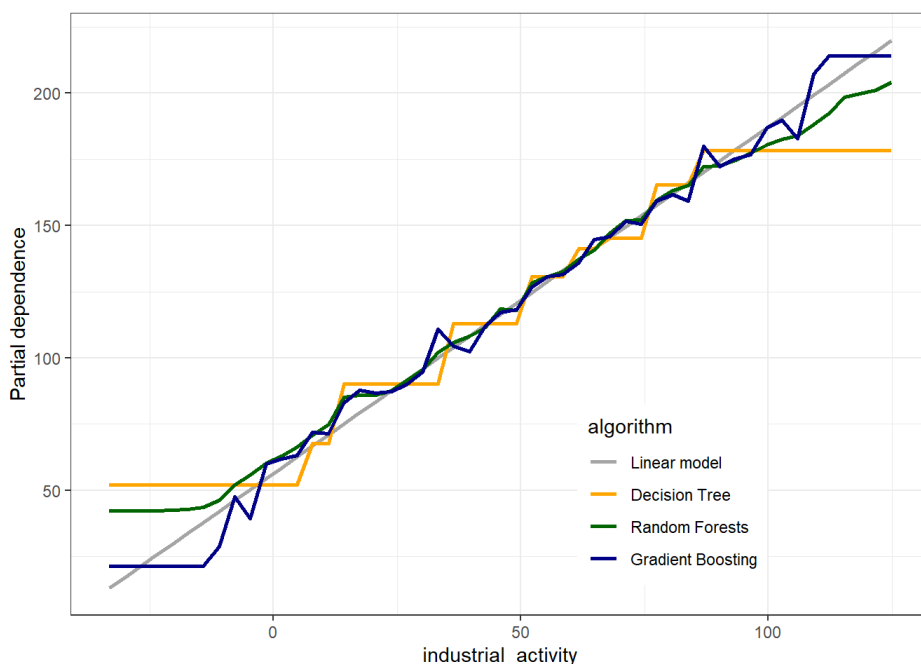
Another popular model interpretation method is the partial dependence plot. It gives information about how a predictor variable and the predicted response variable are linked to each other.


```
# Combine partial dependence data for the industrial_activity predictor across all models
pdp1 <- rbind(
  model.lm %>% partial(pred.var=c("industrial_activity")) %>% cbind(., algorithm = "Linear model"),      # PDP from Linear model
  model.cart %>% partial(pred.var=c("industrial_activity"), approx=F) %>% cbind(., algorithm = "Decision Tree"), # PDP from decision tree (exact values)
  model.rf %>% partial(pred.var=c("industrial_activity"), approx=F) %>% cbind(., algorithm = "Random Forests"), # PDP from random forest
  model.gbm %>% partial(pred.var=c("industrial_activity"), approx=F) %>% cbind(., algorithm = "Gradient Boosting") # PDP from gradient boosting
)

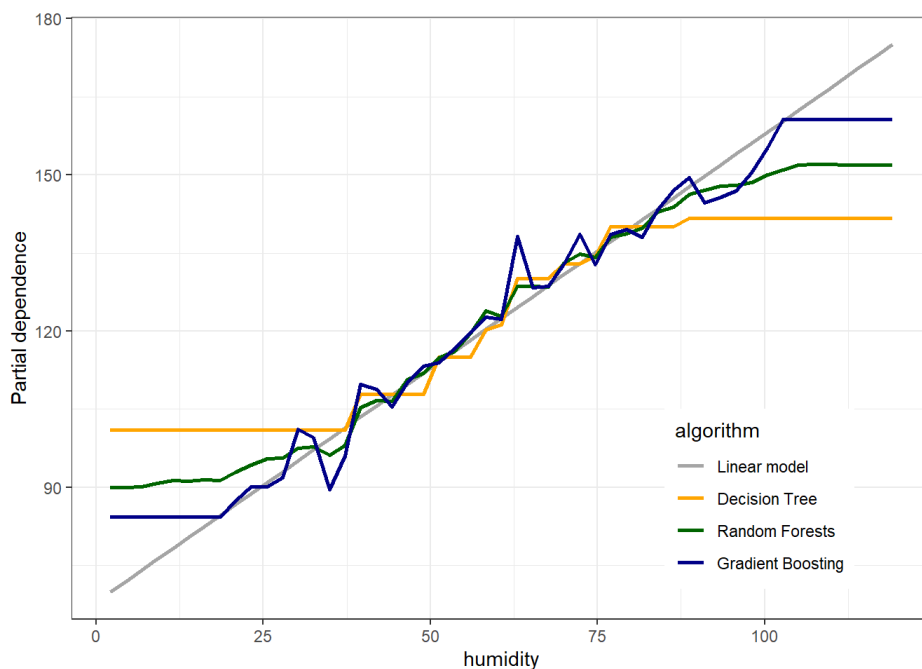
# set factor levels so the order of algorithms is consistent in plots
pdp1$algorithm <- factor(pdp1$algorithm, levels=c("Linear model","Decision Tree","Random Forests","Gradient Boosting"))

# Combine partial dependence data for the humidity predictor across all models
pdp2 <- rbind(
  model.lm %>% partial(pred.var=c("humidity")) %>% cbind(., algorithm = "Linear model"),
  model.cart %>% partial(pred.var=c("humidity"), approx=F) %>% cbind(., algorithm = "Decision Tree"),
  model.rf %>% partial(pred.var=c("humidity"), approx=F) %>% cbind(., algorithm = "Random Forests"),
  model.gbm %>% partial(pred.var=c("humidity"), approx=F) %>% cbind(., algorithm = "Gradient Boosting")
)
pdp2$algorithm <- factor(pdp2$algorithm, levels=c("Linear model","Decision Tree","Random Forests","Gradient Boosting"))
```

```
Fig05a <-
ggplot(pdp1, aes(x = `industrial_activity`, y = yhat, color = algorithm)) +
  geom_line(size = 1) +
  scale_color_manual(values = c("Linear model" = "darkgrey",
                                "Decision Tree" = "orange",
                                "Random Forests" = "darkgreen",
                                "Gradient Boosting" = "darkblue")) +
  ylab("Partial dependence") +
  theme_bw() +
  theme(legend.position = c(0.7, 0.2))
Fig05a # plot PDP curves for industrial_activity across models
```



```
Fig05b <-
ggplot(pdp2, aes(x = `humidity`, y = yhat, color = algorithm)) +
  geom_line(size = 1) +
  scale_color_manual(values = c("Linear model" = "darkgrey",
                                "Decision Tree" = "orange",
                                "Random Forests" = "darkgreen",
                                "Gradient Boosting" = "darkblue")) +
  ylab("Partial dependence") +
  theme_bw() +
  theme(legend.position = c(0.8, 0.2))
Fig05b # plot PDP curves for humidity across models
```



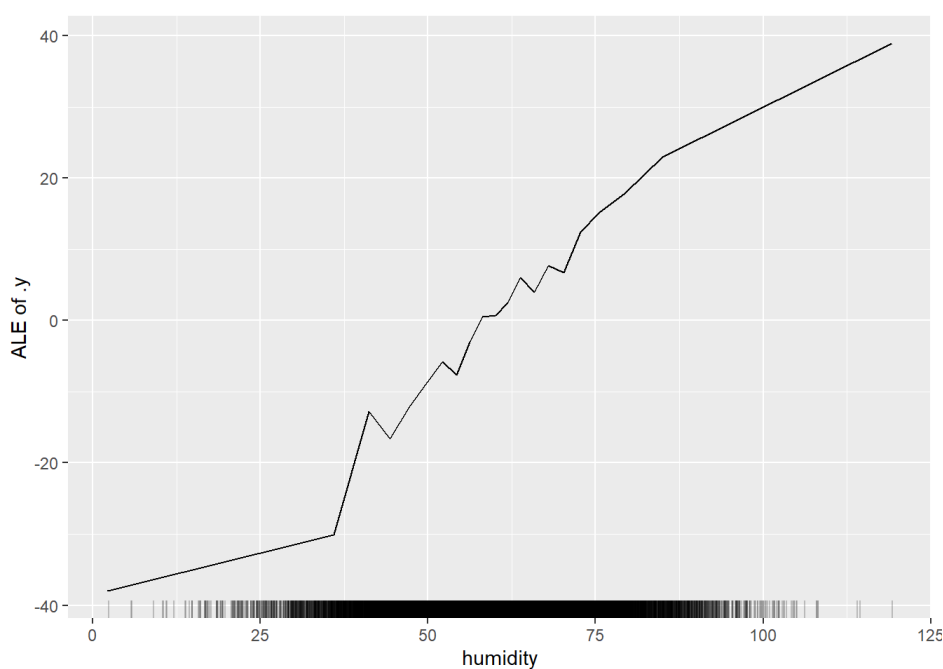
PDPs allow detecting relationships between predictors and response variables. Indeed, in the environmental and earth system sciences, partial dependence plots are a popular choice. However, when dealing with highly correlated variables, Accumulated local effect (ALE) plots are considered more reliable than PDP, as PDP assumes independence of variables which often cannot be guaranteed in environmental sciences.

Here is an example to draw an ALE plot.

```
# Define a generic prediction function for caret models
pred.fun <- function(model, newdata) predict(model, newdata)

# For the random forest
predictor_gbm <- Predictor$new(
  model      = model.gbm,
  data       = data_train[, -which(names(data_train) == "AQI")],
  y          = data_train$AQI,
  predict.fun = pred.fun
)

ale_gbm_humidity <- FeatureEffect$new(
  predictor = predictor_gbm,
  feature   = "humidity",
  method    = "ale"
)
plot(ale_gbm_humidity)
```



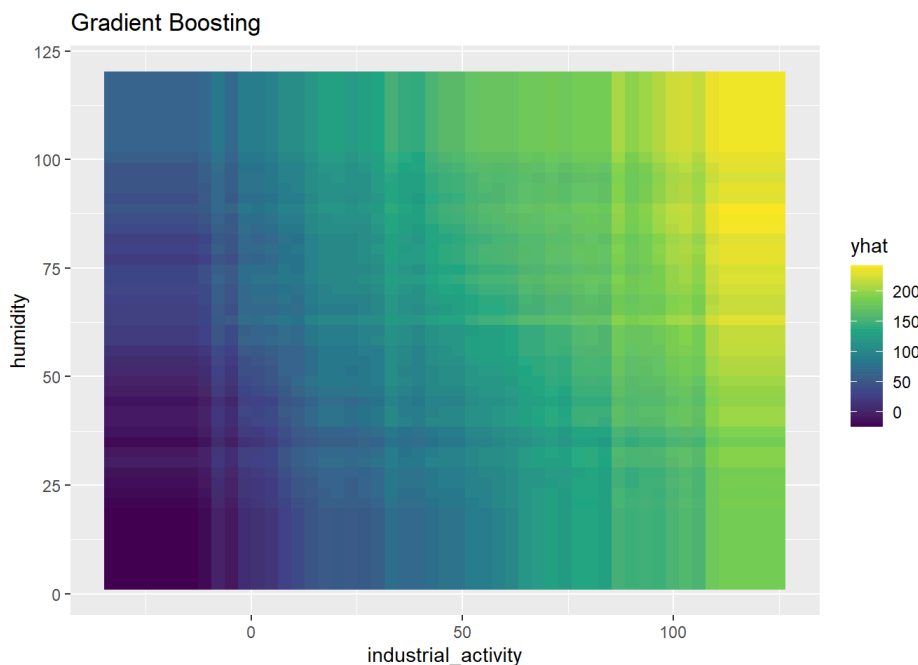
5.3 Variable interactions

Domain knowledge about air quality may suggest a few plausible interactions:

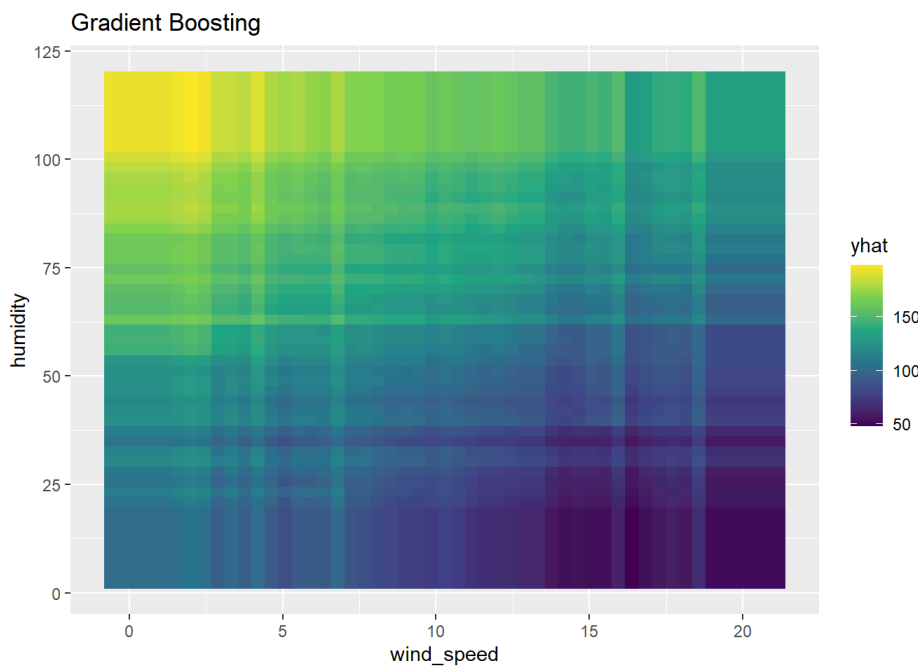
- **Industrial activity × Humidity:** High humidity can trap particulate matter and slow dispersion, so the effect of industrial emissions may depend on ambient moisture.
- **Industrial activity × Wind speed:** Increased wind can disperse pollutants, so emissions might be less impactful when winds are strong.
- **Humidity × Temperature:** Temperature and humidity together influence chemical reactions that create secondary pollutants like ozone.

We can use 2-Way interaction PDPs to detect variable interaction effects. For this exploration, we select gradient boosting model, because it ranked high in the performance metric and allows to detect non-linear interaction effects.

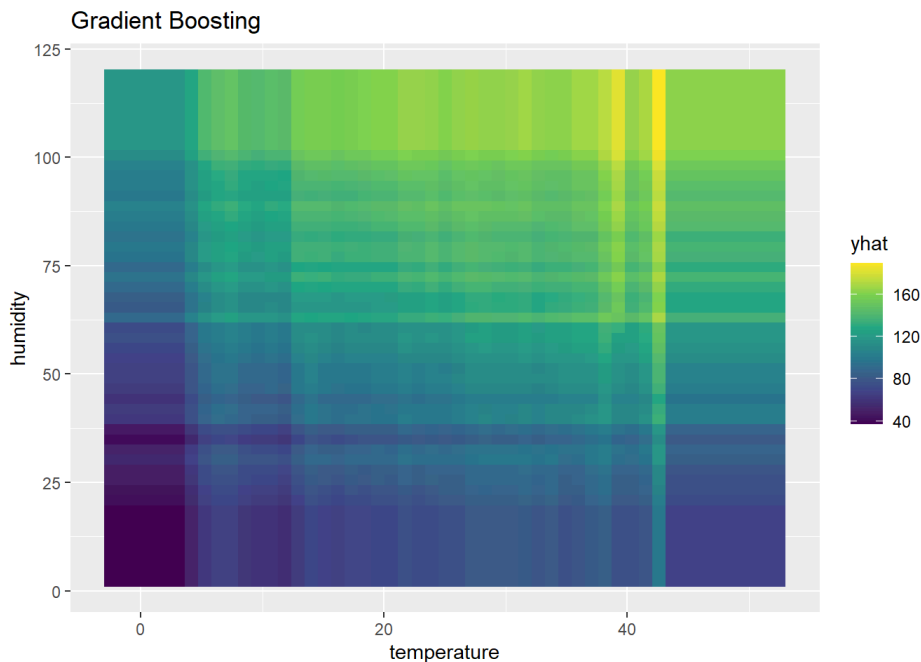
```
pdp2w1 <- model.gbm %>% partial(pred.var=c("industrial_activity", "humidity"), approx=T, n.trees=500) %>% autoplot + labs(title="Gradient Boosting")
pdp2w1
```



```
pdp2w2 <- model.gbm %>% partial(pred.var=c("wind_speed", "humidity"), approx=T, n.trees=500) %>% autoplot + labs(title="Gradient Boosting")
pdp2w2
```



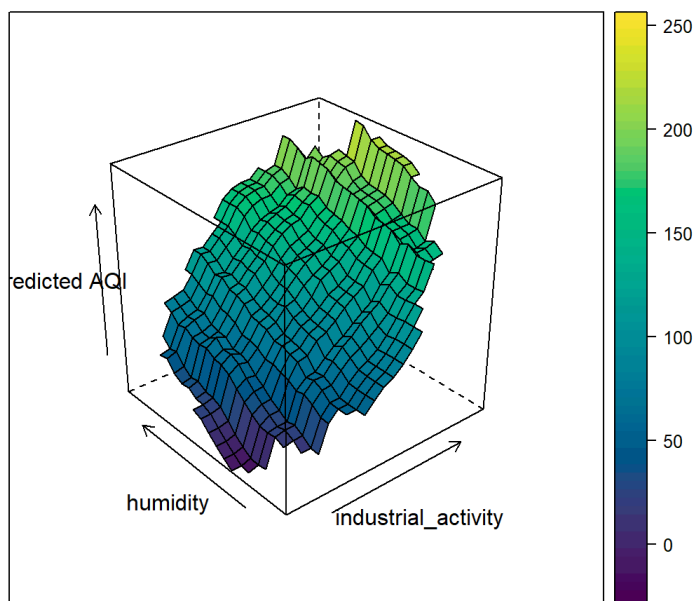
```
pdp2w3 <- model.gbm %>% partial(pred.var=c("temperature", "humidity"), approx=T, n.trees=500) %>% autoplot + labs(title="Gradient Boosting")
pdp2w3
```



Alternatively, we can draw a 3D PDP.

```
# Compute a 3-D PDP for industrial_activity × humidity
pdp_gbm_interact <- partial(
  object      = model.gbm,
  pred.var    = c("industrial_activity", "humidity"),
  n.trees     = model.gbm$bestTune$n.trees, # use the best number of trees found during training
  chull       = TRUE,                      # restrict grid to the data's convex hull
  approx      = TRUE,                      # set to TRUE for faster approximations on large data
  grid.resolution = 25                     # Less grids per dimension
)

# Plot as a wireframe surface
plotPartial(
  pdp_gbm_interact,
  levelplot = FALSE, # FALSE gives a wireframe surface instead of a contour heatmap
  drape      = TRUE,  # add a colour gradient over the surface
  zlab       = "Predicted AQI"
)
```



6 Further literature on learning machine learning

Textbooks

- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *Moving beyond linearity*. In *An Introduction to Statistical Learning: with Applications in Python* (pp. 289–329). Cham: Springer International Publishing.

- Molnar, C. (2020). *Interpretable Machine Learning*. Lulu.com. <https://christophm.github.io/interpretable-ml-book/>
(<https://christophm.github.io/interpretable-ml-book/>)

Case study example

- Ryo, M. (2022). Explainable artificial intelligence and interpretable machine learning for agricultural data analysis. *Artificial Intelligence in Agriculture*, 6, 257–265. (**Basis for the tutorial code**)

Review (Earth & environmental sciences)

- Schiller, J., Stiller, S., & Ryo, M. (2025). Artificial intelligence in environmental and Earth system sciences: explainability and trustworthiness. *Artificial Intelligence Review*, 58(10), 1–23.